

1.8 Packages and Imports

The Java language takes a general and useful approach to the organization of classes into programs. Every stand-alone public class defined in Java must be given in a separate file. The file name is the name of the class with a `.java` extension. So a class declared as **public class** `Window` is defined in a file `Window.java`. That file may contain definitions for other stand-alone classes, but none of them may be declared with public visibility.

To aid in the organization of large code repository, Java allows a group of related type definitions (such as classes and enums) to be grouped into what is known as a **package**. For types to belong to a package named *packageName*, their source code must all be located in a directory named *packageName* and each file must begin with the line:

```
package packageName;
```

By convention, most package names are lowercased. For example, we might define an architecture package that defines classes such as `Window`, `Door`, and `Room`. Public definitions within a file that does not have an explicit **package** declaration are placed into what is known as the **default package**.

To refer to a type within a named package, we may use a fully qualified name based on dot notation, with that type treated as an attribute of the package. For example, we might declare a variable with `architecture.Window` as its type.

Packages can be further organized hierarchically into **subpackages**. Code for classes in a subpackage must be located within a subdirectory of the package's directory, and qualified names for subpackages rely on further use of dot notation. For example, there is a `java.util.zip` subpackage (with support for working with ZIP compression) within the `java.util` package, and the `Deflater` class within that subpackage is fully qualified as `java.util.zip.Deflater`.

There are many advantages to organizing classes into packages, most notably:

- Packages help us avoid the pitfalls of name conflicts. If all type definitions were in a single package, there could be only one public class named `Window`. But with packages, we can have an `architecture.Window` class that is independent from a `gui.Window` class for graphical user interfaces.
- It is much easier to distribute a comprehensive set of classes for other programmers to use when those classes are packaged.
- When type definitions have a related purpose, it is often easier for other programmers to find them in a large library and to better understand their coordinated use when they are grouped as a package.
- Classes within the same package have access to any of each others' members having **public**, **protected**, or default visibility (i.e., anything but **private**).

Import Statements

As noted on the previous page, we may refer to a type within a package using its fully qualified name. For example, the `Scanner` class, introduced in Section 1.6, is defined in the `java.util` package, and so we may refer to it as `java.util.Scanner`. We could declare and construct a new instance of that class in a project using the following statement:

```
java.util.Scanner input = new java.util.Scanner(System.in);
```

However, all the extra typing needed to refer to a class outside of the current package can get tiring. In Java, we can use the **import** keyword to include external classes or entire packages in the current file. To import an individual class from a specific package, we type the following at the beginning of the file:

```
import packageName.className;
```

For example, in Section 1.6 we imported the `Scanner` class from the `java.util` package with the command:

```
import java.util.Scanner;
```

and then we were allowed to use the less burdensome syntax:

```
Scanner input = new Scanner(System.in);
```

Note that it is illegal to import a class with the above syntax if a similarly named class is defined elsewhere in the current file, or has already been imported from another package. For example, we could not simultaneously import both `architecture.Window` and `gui.Window` to use with the unqualified name `Window`.

Importing a Whole Package

If we know we will be using many definitions from the same package, we can import all of them using an asterisk character (*) to denote a wildcard, as in the following syntax:

```
import packageName.*;
```

If a locally defined name conflicts with one in a package being imported in this way, the locally defined one retains the unqualified name. If there is a name conflict between definitions in two different packages being imported this way, *neither* of the conflicting names can be used without qualification. For example, if we import the following hypothetical packages:

```
import architecture.*;    // which we assume includes a Window class
import gui.*;             // which we assume includes a Window class
```

we must still use the qualified names `architecture.Window` and `gui.Window` in the rest of our program.